

Final Report: CPU Scheduling Simulator

Course: ICS 433
Submission: Phase 3 Final Submission
Date: May 4, 2026

Team Members

Name	ID	Contribution
Abdulaziz Alomair	202185570	Simulation engine, event queue, shared simulator contracts
Feras Alharbi	202258300	Scheduling algorithms and scheduler tests
Faisal Alyousef	202171870	I/O parser, metrics, output tables, Gantt chart, and main integration

1. Introduction

The CPU Scheduling Simulator is a mini operating-system project written in C. It models how an operating system chooses which process should run on the CPU when multiple processes are ready. The simulator reads a workload file, runs one scheduling algorithm, and prints an ASCII Gantt chart plus timing statistics.

The final project implements the planned core system: process loading, event-driven simulation, scheduler policies, metrics, and formatted output.

2. Problem Statement and Motivation

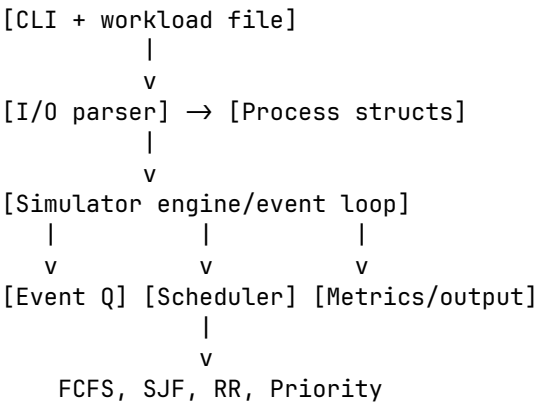
CPU scheduling is one of the central responsibilities of an operating system. Different algorithms can produce different response time, waiting time, turnaround time, CPU utilization, and throughput results.

The motivation for this project is to make those tradeoffs visible with a small, inspectable simulator. Instead of only studying formulas, the simulator executes workloads and shows how FCFS, SJF, Round Robin, and Priority scheduling behave.

3. System Design

The simulator is event-driven. The clock advances directly to the next important event rather than moving one time unit at a time. This design keeps the system efficient and matches how scheduling decisions happen around process arrivals, CPU completion, I/O completion, and preemption.

Architecture Diagram



Module Explanation

Module	Responsibility
src/include/	Shared process, event, state, and algorithm definitions
src/io/	Parses command-line arguments and workload files
src/engine/	Runs the event loop and manages arrivals, CPU completion, I/O completion, preemption, and context switches
src/scheduler/	Provides a scheduler interface and implements FCFS, SJF, RR, and Priority ready queues
src/metrics/	Records execution data and prints response, waiting, turnaround, utilization, throughput, and context-switch metrics
src/output/	Prints the run header, Gantt chart, and final statistics
tests/	Contains focused module and integration tests

4. Implementation Details

Event Queue

The event queue is implemented as a min-heap sorted by event timestamp. It supports insertion, pop, empty check, and dynamic capacity growth.

Simulator Engine

The simulator seeds one arrival event for each process. During execution, it handles:

- `EVENT_ARRIVAL`: process enters the ready queue.
- `EVENT_CPU_DONE`: process finishes a CPU burst and either waits for I/O or terminates.
- `EVENT_IO_DONE`: process returns to the ready queue.
- `EVENT_PREEMPTION`: Round Robin time slice expires and the process is requeued.

The engine also tracks CPU busy time, context switches, response time, and ready-queue waiting time.

Scheduling Algorithms

Algorithm	Implementation
FCFS	FIFO ready queue based on arrival/enqueue order
SJF	Non-preemptive ordering by next CPU burst length
Round Robin	FIFO ordering with simulator-managed quantum preemption
Priority	Non-preemptive ordering by lower numeric priority value

SJF and Priority use stable tie-breaking by enqueue sequence and then process ID.

I/O, Metrics, and Output

The workload parser accepts comments and blank lines. Each process has one header line and one burst line. The output layer prints the algorithm, process count, Gantt chart, per-process table, and system-wide statistics.

5. Tools and Technologies

- C
- GCC
- Make
- Git and GitHub
- Markdown and Chromium for PDF report generation

6. Results and Testing

Build Result

```
$ make all
gcc -Wall -Wextra -g -Isrc/include -o scheduler ...
gcc -Wall -Wextra -g -Isrc/include -o test_event_queue ...
gcc -Wall -Wextra -g -Isrc/include -o test_scheduler ...
gcc -Wall -Wextra -g -Isrc/include -o test_simulator ...
```

Test Cases

Test	Purpose	Result
test_event_queue	Verifies chronological min-heap pop order	Passed
test_scheduler	Verifies FCFS, SJF, RR, Priority ordering and reset behavior	Passed
test_simulator	Verifies scheduler integration, RR preemption, non-preemption, and I/O re-entry	Passed

Terminal captures:

```
$ ./test_event_queue
--- Running Event Queue Tests ---
Pop 1 passed (Timestamp: 2)
Pop 2 passed (Timestamp: 8)
Pop 3 passed (Timestamp: 15)
Empty check passed
All Event Queue tests passed.

$ ./test_scheduler
--- Running Scheduler Tests ---
All scheduler tests passed.

$ ./test_simulator
--- Running Simulator + Scheduler Tests ---
Simulator integration tests passed.
```

Algorithm Output Summary

The final sample workload contains five processes with CPU and I/O bursts. All four algorithms run successfully.

Algorithm	End Time	CPU Utilization	Avg Response	Avg Waiting	Avg Turnaround	Context Switches
FCFS	76	82.89%	9.40	31.60	49.00	12
SJF	74	85.14%	18.00	34.80	52.20	12
Priority	74	85.14%	18.00	29.60	47.00	12
Round Robin, q=4	82	76.83%	7.60	44.20	63.20	20

Example Round Robin output:

```
$ ./scheduler workload.txt rr 4
Algorithm : Round Robin (RR)
Quantum   : 4
Processes : 5

GANTT CHART
|P1 |P2 |P1 |P3 |P4 |P5 |P2 |P1 |P3 |P4 |P5 |P2 |P1 |P3 |P5 |P4 |P1 |P4 |P1 |P4 |
0   5   10  13  18  22  27  32  37  42  47  51  55  57  59  63  66  71  76  81  82

SYSTEM-WIDE STATISTICS
Total processes      : 5
Simulation end time  : 82
CPU busy time        : 63
CPU utilization      : 76.83%
```

Throughput	: 0.0610 proc/unit
Context switches	: 20
Avg response time	: 7.60
Avg waiting time	: 44.20
Avg turnaround time	: 63.20

7. Challenges and Solutions

Challenge	Solution
Keeping team modules independent	Defined shared hooks and headers between engine, scheduler, metrics, and output
Supporting multiple algorithms cleanly	Used one scheduler API with algorithm-specific enqueue behavior
Round Robin preemption	Kept quantum/preemption timing in the simulator while ready-queue order stays in scheduler
Correct metrics after I/O and preemption	Tracked ready-queue entry time and accumulated waiting time at dispatch
Producing readable output	Added formatted statistics and an ASCII Gantt chart

8. Future Improvements

- Add preemptive SJF and preemptive Priority variants.
- Add JSON/CSV output export for easier analysis.
- Add more workload validation and detailed parse error locations.
- Add automated end-to-end tests that verify full CLI output.
- Add optional idle-time segments to the Gantt chart.

9. Conclusion

The final project is a complete working CPU scheduling simulator with integrated workload parsing, scheduling algorithms, event-driven execution, metrics, Gantt chart output, tests, and documentation. The project demonstrates core operating-systems scheduling behavior and provides a foundation for experimenting with additional scheduling policies.